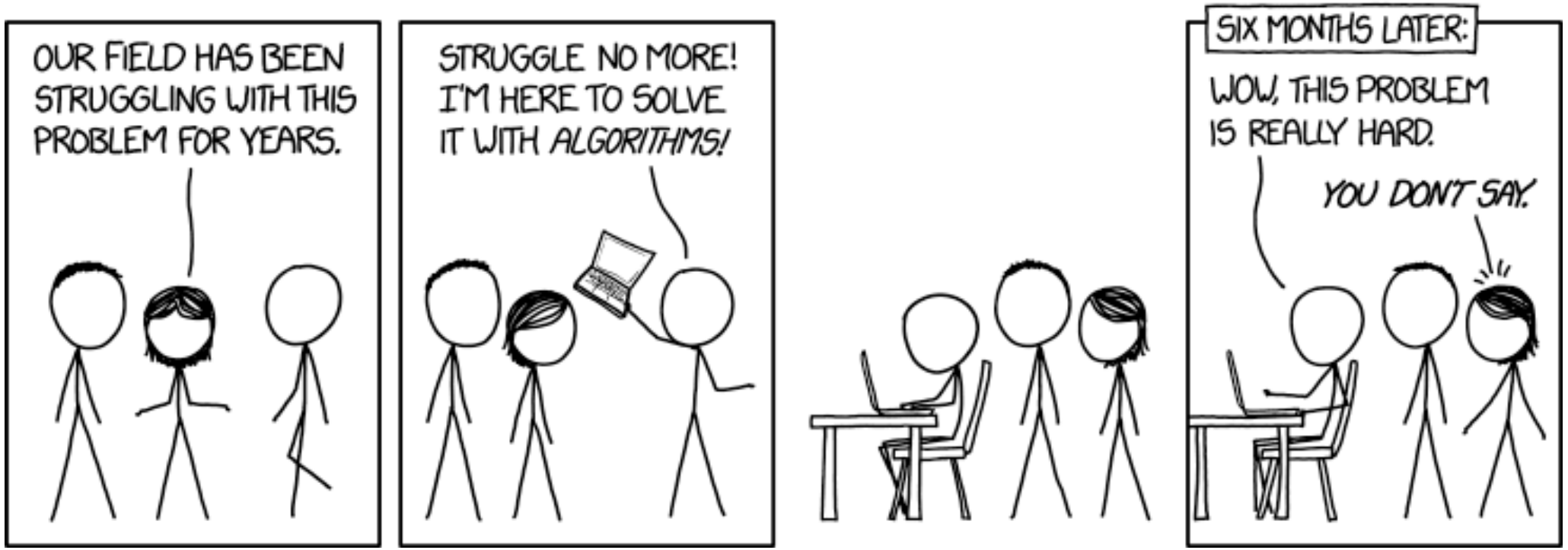
Datastructuren en Algoritmiek

Steven Mooij

HBO-ICT



Bron: https://xkcd.com

DE HAAGSE HOGESCHOOL

Voorbereiding

- Bestudeer deze slides en de referenties waarnaar verwezen wordt.
- Maak, voor zover mogelijk, de opdracht van het bij deze les behorend practicum.

Leerdoelen Datastructuren en Algoritmiek

1) De student schrijft een eenvoudig algoritme waarmee een gegeven probleem wordt opgelost, gebruik makend van recursie, bestaande algoritmen en datastructuren.

2) De student schrijf een eenvoudige datastructuur waarmee een gegeven probleem wordt opgelost, gebruik makend van recursie, bestaande algoritmen en datastructuren.

3) De student bepaalt de tijdscomplexiteit van een zelf geschreven of bestaand algoritme en kan hierbij gebruik maken van elementaire formules en de ‘grote-O-notatie’.

Steven Mooij

HBO-ICT
DE HAAGSE HOGESCHOOL

Na vandaag…

Weet/ken je…

wat een logaritme is

wat een algoritme is

wat ‘recursief programmeren’ inhoudt

HC + WC

Kun je…

een onderbouwde inschatting maken van de ‘tijdscomplexiteit’ van een algoritme

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Algoritme

Een algoritme is een stappenplan bestaande uit een eindige hoeveelheid regels in vaste volgorde om tot een oplossing te komen en het einddoel te bereiken. (bron: https://nl.wikipedia.org)

Steven Mooij

HBO-ICT
DE HAAGSE HOGESCHOOL

Algoritme (voorbeeld)

Doel: tomatensoep

1) Dompel 1 kg tomaten 15 seconden in kokend water.

2) Pel de tomaten.

3) Pel 3 uien.

4) Fruit de uien.

5) Kook de uien en de tomaten 15 minuten in vleesbouillon.

6) Pureer dit mengsel met een staafmixer.

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Algoritme (kenmerken)

Correct
Het behaald het beoogde resultaat

Efficiënt
Kan het ‘beter’ (bijvoorbeeld sneller, of minder geheugen)?

Bepaald
Elke instructie is duidelijk en ondubbelzinnig

Eindig
Het algoritme stopt een keer

Algoritme

Een aantal technieken die van invloed zijn op de efficiëntie:

- **Brute force**: We proberen gewoon alles te berekenen.
- **Divide & Conquer**: Verklein het probleem, los op en combineer.
- **Greedy technique:** Elke stap bepaald het optimum zonder blik op het gehele plaatje.
- **Dynamic Programming:** Divide & Conquer, maar geeft de berekende oplossing door via een datastructure.
- **Generics:** Laat evolutie bepalen wat de beste oplossing is.
- **Backtracking**: Stapgewijs een oplossing zoeken. Werkt een bepaalde combinatie van stappen niet, doe dan een stapje terug.
- …

Ook goed om rekening mee te houden:

- Complexiteit
- Recursie vs iteratief
- Serieel vs distributed algorithms

Algoritme (voorbeeld)

Doel: een pak kaarten sorteren in een gewenste volgorde (bijvoorbeeld 2 t/m 10, boer, vrouw, heer, aas en klaveren, harten, schoppen, ruiten).

1) Schud de kaarten zodat ze in een willekeurige volgorde liggen.

2) Als de kaarten niet in de gewenste volgorde liggen, spring dan naar stap 1.

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Tijdscomplexiteit

De efficiëntie van een algoritme wordt vaak uitgedrukt in het maximum aantal uitgevoerde stappen dat een algoritme nodig heeft om tot een oplossing te komen. Dit wordt ook wel de tijdscomplexiteit genoemd.

Meestal wordt deze uitgedrukt in de grote-O-notatie (zie volgende les).

De tijd die 1 stap kost is niet relevant.

Ruimtecomplexiteit is soms ook relevant, maar wordt in dit vak buiten beschouwing gelaten.

Steven Mooij

HBO-ICT
DE HAAGSE HOGESCHOOL

Oefening

Doel: Een zo groot mogelijke stapel enveloppen in elkaar steken. De hoogte en breedte van elke envelop is bekend. Een envelop past in een andere envelop als de hoogte en de breedte kleiner is. Enveloppen mogen niet worden gedraaid.

Op Brightspace staat een bestand ‘enveloppen.txt’ met hoogtes en breedtes van 50 enveloppen.

Maak een zo groot mogelijke stapel.

Verzin een algoritme dat voor een willekeurige verzameling enveloppen de grootst mogelijke stapel bepaalt.

Verzin een efficiënt algoritme dat voor een willekeurige verzameling enveloppen de grootst mogelijke stapel bepaalt.

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Tijdscomplexiteit

```
def calculate(n):
    result = 0;
    for a in range(0, n):
        result = foo(result)        # foo(result) = O(1)
    for b in range (0, n):
        result = bar(result)        # bar(result) = O(1)
```

(maximaal) 2 * n stappen → O(n)

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Tijdscomplexiteit

```
def calculate(n):
    result = 0;
    for a in range(0, n):
        for b in range (0, n):
            result = foo(result)      # foo(result) = O(1)
```

(maximaal) n * n stappen → $O(n^2)$

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Tijdscomplexiteit

```
def calculate(n):
    result = n;
    for a in range(0, 1000):
        result = foo(result)      # foo(result) = O(1)
```

(maximaal) 1 stap → O(1)

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Tijdscomplexiteit

```
def search(element, list):
    result = False;
    for x in list:
        result |= element == x
    return result
```

maximaal len(list) stappen → O(len(list))

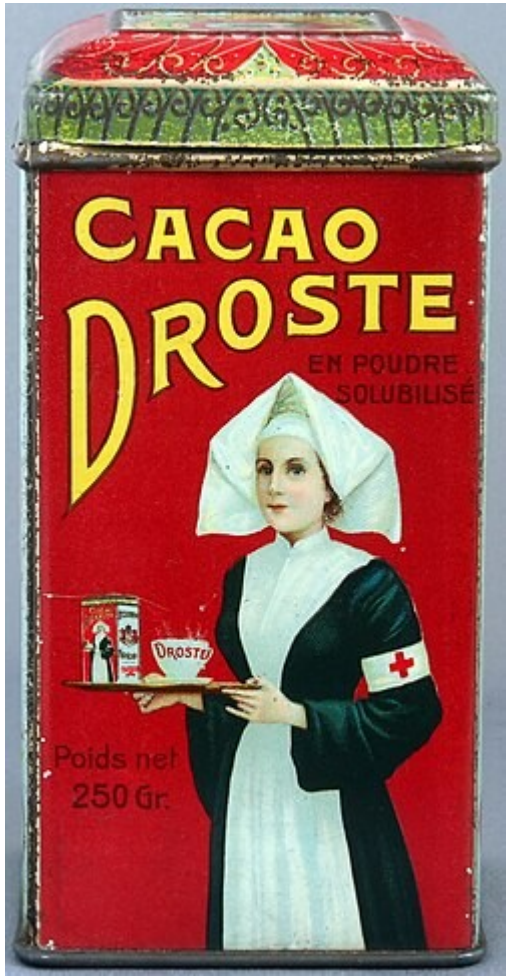
Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Recursie

Een methode waar de oplossing van een probleem afhangt van oplossingen van kleinere identieke problemen (bron: https://www.wikipedia.nl)

Een definitie (van een algoritme of datastructuur) maakt (direct of indirect) gebruik van de definitie zelf.

Alles wat recursief opgelost kan worden kan ook iteratief opgelost worden (en vice versa). Recursie begrijpen kost wellicht tijd, maar is de moeite waard. Er zijn problemen die (veel) eenvoudiger recursief op te lossen zijn dan iteratief. Bijvoorbeeld ‘divide and conquer’ algoritmen zoals ‘mergesort’.



Recursie

Voorbeeld: bereken n! (1 * 2 * … * n, 0! = 1)

```
def faculty (n):
    return 1 if n == 0 else n * faculty(n - 1)
```

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Recursie

Voorbeeld: bepaal of n even is

```
def even(n):
    return True if n == 0 else odd(n - 1)

def odd(n):
    return False if n == 0 else even(n - 1)
```

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Logaritme

De logaritme van een getal is de exponent waartoe een vast getal, het zogenaamde grondtal, moet worden verheven om dat eerste getal als resultaat te verkrijgen. (Wikipedia)

De logaritme voor het grondtal a van een getal x is de macht waartoe men het grondtal a moet verheffen om x als uitkomst te krijgen, dus:

$q = \log_a (x) \Longleftrightarrow a^q = x$

Zowel het grondtal a als het argument x moeten groter zijn dan 0; bovendien mag het grondtal a niet gelijk zijn aan 1.

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Logaritme

$$7^{?} = 96.889.010.407$$

$$? = \log_{7}(96.889.010.407) = 13$$

$$3{,}41^{?} = 567{,}989$$

$$? = \log_{3{,}41}(567{,}989) = 5{,}17$$

Logaritme

$2^3 = 8 \Leftrightarrow \log_2(8) = 3$

$3^2 = 9 \Leftrightarrow \log_3(9) = 2$

$4^3 = 64 \Leftrightarrow \log_4(64) = 3$

$\log_2(1.024) = 10 \Leftrightarrow 2^{10} = 1.024$

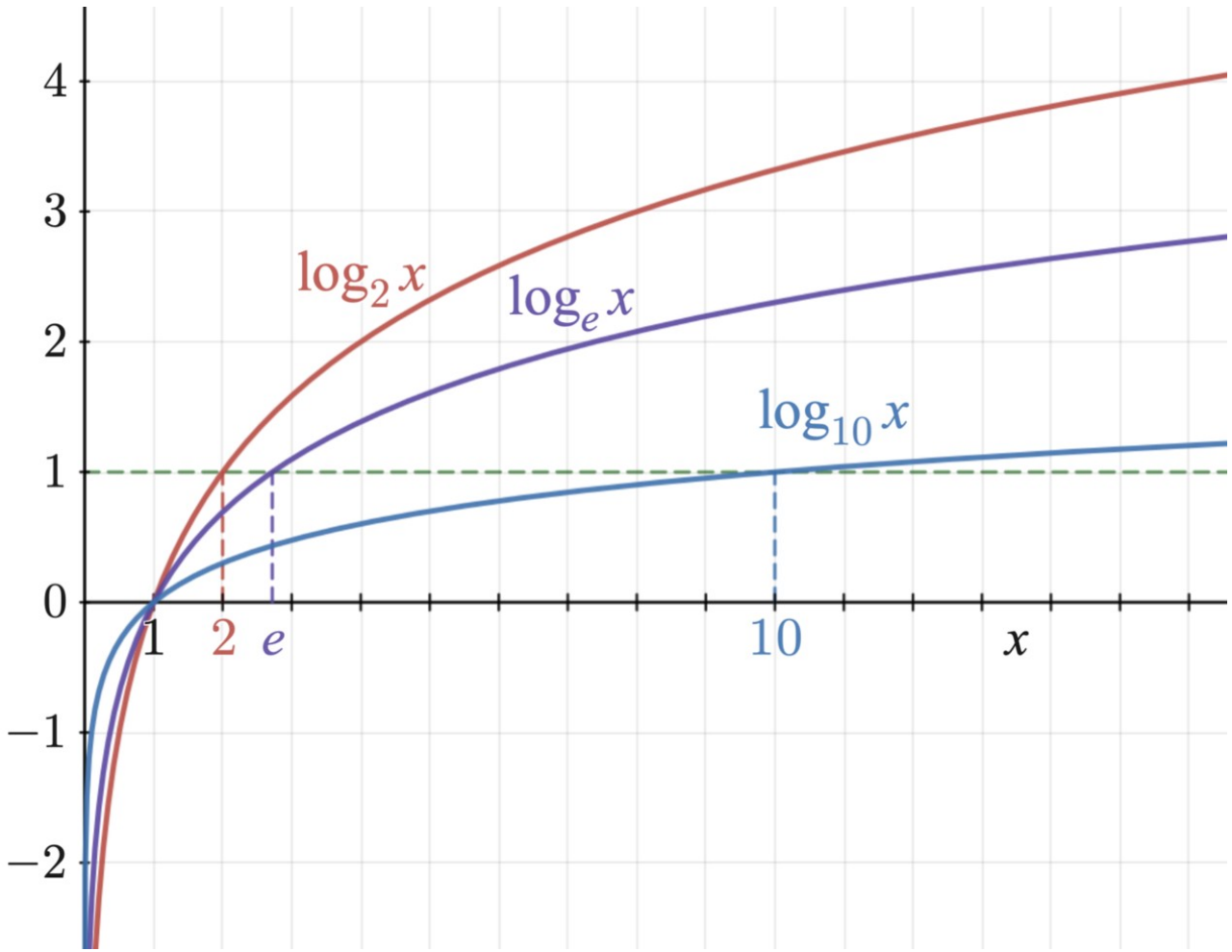
$\log_5(25) = 2 \quad (\Leftrightarrow 5^2 = 25)$

$\log_{10}(1.000.000) = 6 \quad (\Leftrightarrow 10^6 = 1.000.000)$

$\log_2(128) = 7 \quad (\Leftrightarrow 2^7 = 128)$

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL

Logaritme



Bron: https://nl.wikipedia.org

Nuttige links

https://nl.wikipedia.org/wiki/Logaritme

https://nl.khanacademy.org/math/algebra-basics

https://inventwithpython.com/recursion/

https://codeforces.com/

https://leetcode.com/

Steven Mooij

HBO-ICT

DE HAAGSE HOGESCHOOL

Na vandaag…

Weet/ken je…

wat een logaritme is

wat een algoritme is

wat ‘recursief programmeren’ inhoudt

HC + WC

Kun je…

een onderbouwde inschatting maken van de ‘tijdscomplexiteit’ van een algoritme

Steven Mooij
HBO-ICT
DE HAAGSE HOGESCHOOL